Brush

Brush

1. Course Content
2. Program Startup
3. Core Source Code Analysis

1. Course Content

- Run the example program to detect fingers using the mediapipe framework, then select a color and draw the movement trajectory on the image according to the recognized finger movement.

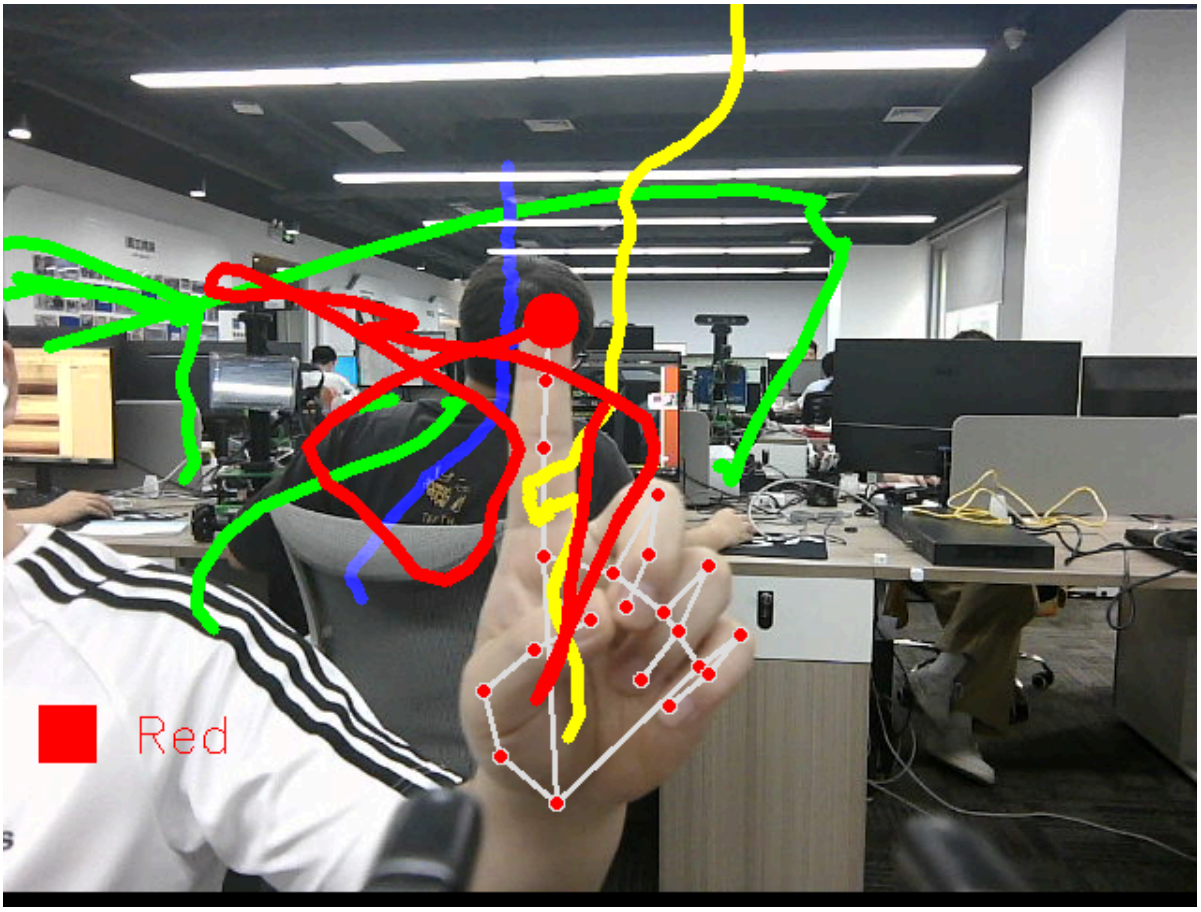
2. Program Startup

- Run the brush example program in the terminal

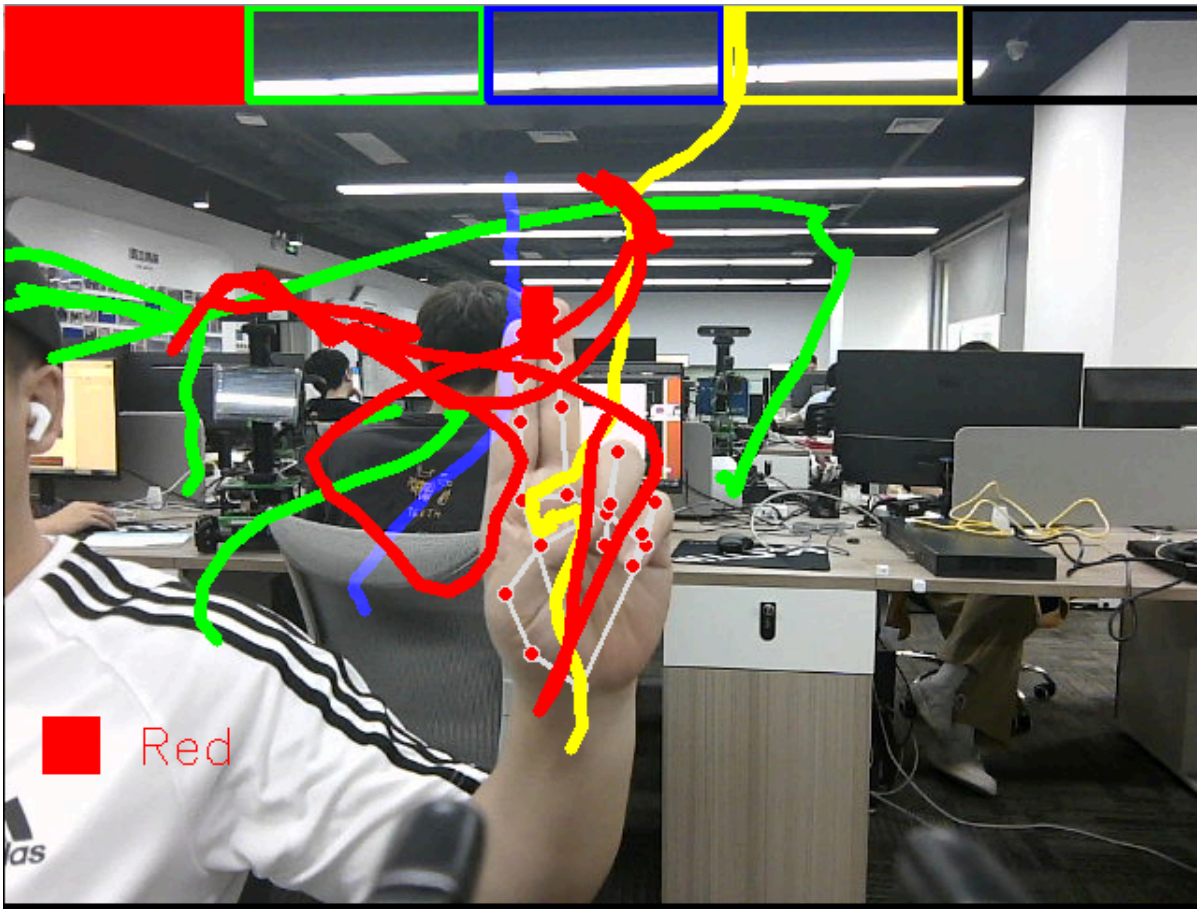
```
ros2 launch yahboomcar_bringup mediapipe_demo.launch.py demo:=VirtualPaint
```

After running, as shown in the figure below, the default fingertip color is red. When the index finger and middle finger of the right hand are put together, it is in the selection state, and a color selection box pops up at the same time. When the two fingertips move to the position of the corresponding color, that color is selected (black is the eraser). When the index finger and middle finger are separated, it is in the drawing state, and you can draw freely on the drawing board.

When the index finger and middle finger are separated, it is in the drawing state.



Put the index finger and middle finger together to enter the color selection mode.



3. Core Source Code Analysis

Program code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/mediapipe/08_VirtualPaint.py
```

- Library files used,

```
import math
import time
import cv2 as cv
import numpy as np
import mediapipe as mp
import rclpy
from rclpy.node import Node
from cv_bridge import CvBridge
from sensor_msgs.msg import Image
from arm_msgs.msg import ArmJoints
import cv2
print("import done")
```

Initialize the data and define the Publisher and Subscriber,

```
def __init__(self,name):
    super().__init__(name)
```

```
    self.xp = self.yp = self.pTime = self.boxx = 0
    self.tipIds = [4, 8, 12, 16, 20]
    #Create a blank image with a resolution of 480*640
    self.imgCanvas = np.zeros((480, 640, 3), np.uint8)
    #Define the thickness of the brush (red, green, blue, yellow)
    self.brushThickness = 5
    #Define the thickness of the eraser (black)
    self.eraserThickness = 100
    #The y-coordinate threshold of the bottom of the color selection box; being
smaller than this value means entering the color selection mode
    self.top_height = 50
    self.Color = "Red"
    #Define the brush color list; the values after it are the rgb values of each
color
    self.ColorList = {
    'Red': (0, 0, 255),
    'Green': (0, 255, 0),
    'Blue': (255, 0, 0),
    'Yellow': (0, 255, 255),
    'Black': (0, 0, 0),
    }
    #Define the ID of each fingertip, later used to determine which fingers are
straight
    self.tipIds = [4, 8, 12, 16, 20]
    #Use the class in the mediapipe library to define a hand object
    self.mpHand = mp.solutions.hands
    self.mpDraw = mp.solutions.drawing_utils
    self.hands = self.mpHand.Hands(
    static_image_mode=False,
    max_num_hands=2,
    min_detection_confidence=0.85,
    min_tracking_confidence=0.5 )
    #Define the properties of the joint connection lines, which will be used in
the subsequent joint point connection function
    self.lmDrawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 0, 255),
thickness=-1, circle_radius=15)
    self.drawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 255, 0),
thickness=10, circle_radius=10)

    self.rgb_bridge = CvBridge()
    #Define the Subscriber for the color image topic
    self.sub_rgb =
self.create_subscription(Image,"/camera/color/image_raw",self.get_RGBImageCallBac
k,100)
```

Color image callback function,

```
def get_RGBImageCallBack(self,msg):
    #Use CvBridge to convert the color image message data into image data
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(msg, "bgr8")
    #Obtain the size of the color image, to make it easier to select the color
based on the fingertip coordinates later
    h, w, c = rgb_image.shape
    frame,lmList = self.findHands(rgb_image, draw=False)
    key = cv2.waitKey(1)
```

```
    if len(lmList) != 0:
        # print(lmList)
        # tip of index and middle fingers
        #Obtain the coordinate values of the fingertips of the index finger and
middle finger
        x1, y1 = lmList[8][1:]
        x2, y2 = lmList[12][1:]
        #Call the fingersUp function to return the list of straight fingers
        fingers = self.fingersUp()
        #If both the index finger and middle finger are straight
        if fingers[1] and fingers[2]:
            # print("Seclection mode")
            if y1 < self.top_height:
                #Determine the selected color according to the coordinates of the
index finger and the size of the image
                if 0 < x1 < int(w / 5) - 1:
                    self.boxx = 0
                    self.Color = "Red"
                if int(w / 5) < x1 < int(w * 2 / 5) - 1:
                    self.boxx = int(w / 5)
                    self.Color = "Green"
                elif int(w * 2 / 5) < x1 < int(w * 3 / 5) - 1:
                    self.boxx = int(w * 2 / 5)
                    self.Color = "Blue"
                elif int(w * 3 / 5) < x1 < int(w * 4 / 5) - 1:
                    self.boxx = int(w * 3 / 5)
                    self.Color = "Yellow"
                elif int(w * 4 / 5) < x1 < w - 1:
                    self.boxx = int(w * 4 / 5)
                    self.Color = "Black"
            cv.rectangle(frame, (x1, y1 - 25), (x2, y2 + 25),
self.ColorList[self.Color], cv.FILLED)
            cv.rectangle(frame, (self.boxx, 0), (self.boxx + int(w / 5),
self.top_height), self.ColorList[self.Color], cv.FILLED)
            #Draw the color selection box for each color
            cv.rectangle(frame, (0, 0), (int(w / 5) - 1, self.top_height),
self.ColorList['Red'], 3)
            cv.rectangle(frame, (int(w / 5) + 2, 0), (int(w * 2 / 5) - 1,
self.top_height), self.ColorList['Green'], 3)
            cv.rectangle(frame, (int(w * 2 / 5) + 2, 0), (int(w * 3 / 5) - 1,
self.top_height), self.ColorList['Blue'], 3)
            cv.rectangle(frame, (int(w * 3 / 5) + 2, 0), (int(w * 4 / 5) - 1,
self.top_height), self.ColorList['Yellow'], 3)
            cv.rectangle(frame, (int(w * 4 / 5) + 2, 0), (w - 1,
self.top_height), self.ColorList['Black'], 3)
            #If the index finger and middle finger are not both straight and the
pixel distance between the two fingertips is greater than 50, it indicates a
spread state, and it enters the drawing mode
        if fingers[1] and fingers[2] == False and math.hypot(x2 - x1, y2 - y1) >
50:
            # print("Drawing mode")
            if self.xp == self.yp == 0: self.xp, self.yp = x1, y1
            #If black, it is the eraser mode and erases the drawn trajectory; if
not, draws the trajectory on the image according to the selected color
            if self.Color == 'Black':
```

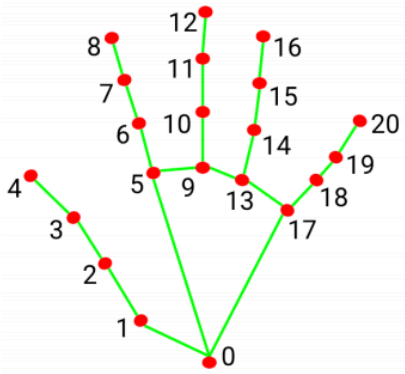
```
                cv.line(frame, (self.xp, self.yp), (x1, y1),
self.ColorList[self.Color], self.eraserThickness)
                cv.line(self.imgCanvas, (self.xp, self.yp), (x1, y1),
self.ColorList[self.Color], self.eraserThickness)
            else:
                cv.line(frame, (self.xp, self.yp), (x1, y1),
self.ColorList[self.Color], self.brushThickness)
                #Draw a line of the selected color on the created canvas
                cv.line(self.imgCanvas, (self.xp, self.yp), (x1, y1),
self.ColorList[self.Color], self.brushThickness)
            cv.circle(frame, (x1, y1), 15, self.ColorList[self.Color], cv.FILLED)
            self.xp, self.yp = x1, y1
        else: self.xp = self.yp = 0
    #Convert the color space of the canvas to a grayscale image for subsequent
image processing
    imgGray = cv.cvtColor(self.imgCanvas, cv.COLOR_BGR2GRAY)
    #Threshold the grayscale image, that is, divide the pixel values of the image
into two categories (such as foreground and background) according to the set
threshold. It is one of the core methods of image binarization.
    _, imgInv = cv.threshold(imgGray, 50, 255, cv.THRESH_BINARY_INV)
    #Convert the color space, turning the grayscale image into the color BGR
image space
    imgInv = cv.cvtColor(imgInv, cv.COLOR_GRAY2BGR)
    #Perform AND and OR operations on the images to combine them
    frame = cv.bitwise_and(frame, imgInv)
    frame = cv.bitwise_or(frame, self.imgCanvas)
    #Draw the currently selected color box and color
    cv.rectangle(frame, (20, h - 100), (50, h - 70), self.ColorList[self.Color],
cv.FILLED)
    cv.putText(frame, self.Color, (70, h - 75), cv.FONT_HERSHEY_SIMPLEX, 0.9, (0,
0, 255), 1)
    cv.imshow('frame', frame)
```

fingersUp function for detecting straight fingers,

```
def fingersUp(self):
    fingers=[]
    # Thumb: determine whether the angle of each joint of the thumb is greater
than 150 degrees; if so, the thumb can be considered straight
    if (self.calc_angle(self.tipIds[0],
                        self.tipIds[0] - 1,
                        self.tipIds[0] - 2) > 150.0) and (
            self.calc_angle(
                self.tipIds[0] - 1,
                self.tipIds[0] - 2,
                self.tipIds[0] - 3) > 150.0): fingers.append(1)
    else:            fingers.append(0)
    # 4 finger: the other four fingers
    for id in range(1, 5):
        #Here it checks whether the y value of the fingertip joint is smaller
than that of the middle joint; if so, it indicates the finger is straight (the
image origin is at the top left, y increases downward, x increases to the right)
        if self.lmList[self.tipIds[id]][2] < self.lmList[self.tipIds[id] - 2][2]:
            fingers.append(1)
        else:
```

```
            fingers.append(0)
    return fingers
```

As shown in the figure below, the ID of each joint of the finger,



0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP

findHands function for detecting the palm,

```
def findHands(self, frame, draw=True):
    #Create a detection list to store the detection results
    self.lmList = []
    img_RGB = cv.cvtColor(frame, cv.COLOR_BGR2RGB)
    #Call the process function in the mediapipe library for image processing;
during init, the self.hands object was created and initialized
    self.results = self.hands.process(img_RGB)
    if self.results.multi_hand_landmarks:
        for handLms in self.results.multi_hand_landmarks:
            if draw: self.mpDraw.draw_landmarks(frame, handLms,
self.mpHand.HAND_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
            else: self.mpDraw.draw_landmarks(frame, handLms,
self.mpHand.HAND_CONNECTIONS)
        #Iterate through the detection results and add them to the self.lmList
list, which represents the id of each detected joint and the xy coordinates of
the joint
        for id, lm in enumerate(self.results.multi_hand_landmarks[0].landmark):
            h, w, c = frame.shape
            cx, cy = int(lm.x * w), int(lm.y * h)
            # print(id, cx, cy)
            self.lmList.append([id, cx, cy])
    return frame, self.lmList
```